

---

# CSE211s Introduction to Embedded Systems

Spring 2023

Sheet 4

Introduction to Embedded Systems

---

1.

For the below ARM assembly code, trace the values as it executes that will be placed into the registers **R4**, **R5**, and **R6**.

```
        MOV R4, #7
        MOV R5, #4
        MOV R6, #4
again   MOV R7, R4
        ADD R4, R5, R4
        MOV R5, R7
        SUBS R6, R6, #1
        BNE again
```

2.

Translate the below C code for counting the 1 bits in **R0** into ARM assembly code, using the registers indicated by the variable names.

```
r3 = 1;
r1 = 0;
while (r3 != 0) {
    if ((r0 & r3) != 0) {
        r1 = r1 + 1;
    }
    r3 = r3 + r3;
}
```

3.

Suppose **R0** contains an integer value  $a$ , and **R1** contains an integer value  $b$  that is more than 0. Write an ARM assembly language fragment that leaves in **R2** the value of  $a^b$  — that is, **R0** raised to the positive power found in **R1**. You may change the values in **R0** and **R1**, but the value for **R2** should be computed reflecting the initial **R0** and **R1** values.

4.

Translate the below C fragment into an equivalent ARM assembly language program, using registers corresponding to the variable names. (As it happens, this code has the effect of placing into **r2** the product of **r0** and **r1**'s initial values.)

```
r2 = 0;
while (r1 != 0) {
    if ((r1 & 1) != 0) {
        r2 += r0;
    }
    r0 <<= 1;
    r1 >>= 1;
}
while (1); // halting loop
```

## 5.

For the below ARM assembly code, trace the values as it executes that will be placed into the registers **PC**, **R0**, **R1**, and **R2**.

```
    MOV R0, #0      ; at addr 0
    MOV R1, #5
    ADD R0, R0, R1
    SUB R1, R1, #1
    CMP R0, #10     ; at addr 16
    MOVGT PC, #28
    MOV PC, #8
    MOV R2, #1
halt B halt        ; at addr 32
```

## 6.

Explain what an ARM processor accomplishes in terms of accessing and changing its registers when it executes a **BEQ** instruction.

## 7.

Suppose **R0** holds the address of the first entry in an array of four-byte integers. Write an ARM assembly language program fragment that places into **R0** the index of the first 0 in that array. (You may assume that there is a 0 in the array.) For example, if the array is <9,7,8,0,6>, and **R0** holds the memory address of the 9, your fragment will place 3 into **R0**, since the 0 is in slot 3 of the array.

## 8.

Suppose **R0** holds the address of the first entry in an array of four-byte integers, and **R1** hold the number of entries in that array. Write an ARM assembly fragment that places into **R0** the largest value of the array. You may change any other registers, and you may assume that the array contains at least one value.

## 9.

Translate the below C code into ARM assembly code, using the registers indicated by the variable names. The C code presumes that **r0** holds the address of the first entry of an array of integer values, and **r1** indicates how many elements the array holds; the code removes all adjacent duplicates from the array.

```
r3 = 1;
for (r2 = 1; r2 < r1; r2++) {
    if (r0[r2] != r0[r2 - 1]) {
        r0[r3] = r0[r2];
        r3 += 1;
    }
}
r1 = r3;
```

## 10.

Suppose we have defined the following C **struct**, which our C compiler has decided to compile with **name** being in the first four bytes and **grade** in the next four bytes.

```
struct student {  
    char *name;  
    int grade;  
};
```

Now suppose **R0** holds the address of an array of twenty 32-bit pointers to **struct student**s; that is, each **R0[i]** holds an address where a **struct student** can be found in memory. Complete the below ARM assembly language fragment as indicated; following completion of the full fragment, **R1** should become the sum of all grades in the course.

```
MOV R1, #0           ; R1 will hold total of all grade  
s seen  
MOV R2, #0           ; R2 is index where we are in arr  
ay  
again                ; load R0[R2]-  
>grade into R3  
ADD R1, R1, R3       ; total += loaded grade  
ADD R2, R2, #1       ; R2 += 1  
CMP R2, #20          ; if R2 < 20, repeat  
BLT again
```

## 11.

Suppose we represent a linked list in memory where each node is eight bytes long, with the node's integer data stored in the first four bytes and the address of the following node found in the following four bytes. The list's last node is marked by having 0 as the address of its following node.

Assuming that **R0** initially holds the address of the list's first node, write an ARM assembly language fragment that counts how many nodes in a linked list have a negative number, placing the number found into **R1**.

## 12.

Find the use of the following assembler directives in the document (DUI0379G\_mdk\_armasm\_user\_guide) on the Internet:

AREA – EQU – DCB – DCW – DCD - SPACE